Predictive Lead Scoring with Machine Learning — Technical Implementation Guide for a B2B Hardware Business (Logitech for Business Profile)

Audience: Data scientists building the model. **Stack:** Salesforce (CRM) + Marketo/HubSpot (marketing automation) + Snowflake (warehouse). **Focus:** feature selection, feature engineering, and modeling — with exact, verified API field names.

TL;DR

- Build a **calibrated binary classifier** (LightGBM/XGBoost as the primary model, regularized logistic regression as the baseline) that predicts the probability of **Lead → Opportunity** (and secondarily Opportunity → Closed Won) using **point-in-time** features from Salesforce, Marketo/HubSpot, and third-party enrichment landed in Snowflake.
- The single most important design decision is **avoiding temporal/target leakage**: build features only from data available at or before the scoring cutoff, and never use post-conversion fields (`IsConverted`, `ConvertedOpportunityId`, `StageName`, `IsWon`, `closedate`) as inputs — they define the target.
- Use only **real API field names** — Salesforce (`Title`, `Industry`, `NumberOfEmployees`, `AnnualRevenue`, `LeadSource`, `Status`), Marketo activity type IDs (1 Visit Webpage, 2 Fill Out Form, 10 Open Email, 11 Click Email), HubSpot properties (`hs_analytics_num_page_views`, `hs_email_open`, `lifecyclestage`) — and write scores back to a Salesforce custom field such as `Predictive_Lead_Score__c`.

Key Findings

- **Vendors converge on a three-layer feature design.** Salesforce Einstein Lead Scoring analyzes past converted leads plus custom and activity data; per Salesforce Help ("Understand How Einstein Scores Your Leads"), *"Einstein models are refreshed every 10 days, or whenever the admin updates how Lead Scoring is configured. Lead scores are updated at least every six hours as needed."* HubSpot's predictive score (`hs_predictivecontactscore_v2`, labeled "Likelihood to close") *"is built using a logistic regression algorithm"* and *"assigns a score between 1 and 100"* representing *"the percentage probability of a contact closing as a customer within the next 90 days."* MadKudu combines a **Customer Fit** model (firmographic/demographic, scores stable over time) with a behavioral **Likelihood to Buy/PQL** model (recomputed several times per day) into a combined grade. This validates a design of **firmographic fit + behavioral engagement + derived recency/velocity**. Madkudu + 5
- **Gradient-boosted trees (XGBoost/LightGBM) are the industry standard for tabular lead scoring**; logistic regression is a strong, interpretable baseline. Boosted models produce uncalibrated probabilities and must be calibrated (Platt/isotonic) because lead scores need to be probability-meaningful.
- **Class imbalance is intrinsic** (conversion often 1–5%). Handle with `scale_pos_weight`/class weights and threshold tuning rather than blind SMOTE; resampling distorts calibration.
- **Feature selection should combine** filter (correlation, chi-square, mutual information, Information Value), embedded (L1/Lasso, tree importance), and SHAP for explainability. An **IV > 0.5 is a red flag for leakage**.

1. Business Context

What predictive lead scoring is. A supervised ML model assigns each lead/contact a probability (0–1) of reaching a defined conversion outcome. Unlike **rule-based scoring** (e.g., Marketo's manual "+10 for a form fill, +5 for an email open" via Change Score smart campaigns, or HubSpot's manual `hubspotscore`), predictive scoring learns weights automatically from historical conversion patterns, captures non-linear interactions, adapts as the model is retrained, and outputs a **calibrated probability** instead of an arbitrary point total.

Why it matters for a B2B hardware company like Logitech for Business. Selling webcams, headsets, and video-conferencing systems (e.g., Rally Bar) into enterprises is a considered, multi-stakeholder purchase. Marketing generates large volumes of MQLs (webinar signups, spec-sheet downloads, "request a demo" forms), but sales capacity is finite. A scoring model lets reps prioritize the ~10–20% of leads that carry most of the pipeline. Firmographic fit (company size, industry, whether they run a competing UC stack) is especially predictive for hardware because deal size scales with the number of rooms/seats to outfit.

How it differs from rule-based scoring. Rule-based: static, human-defined weights, no probability, decays silently when buyer behavior shifts. Predictive: data-derived weights, calibrated probability, measurable AUC/lift, retrainable. Salesforce itself moved Einstein to ML precisely because pre-COVID rule weights (e.g., scoring in-person store visits highly) became wrong as buying moved online.

2. Data Sources and Exact Fields to Use as Features

Organize features into four layers. **All field names below are the real API/field names** from Salesforce, Marketo, and HubSpot documentation.

2A. Demographic/Firmographic Features — Salesforce Lead / Contact / Account

From the **Lead** object (`sforce_api_objects_lead`):

API Name	Type	Feature use
`Title`	string	Seniority/role signal (map to a derived band)
`Industry`	picklist	Vertical fit
`NumberOfEmployees`	int	Company size (strong for hardware seat count)
`AnnualRevenue`	currency	Budget proxy
`Rating`	picklist	Existing qualitative rating
`LeadSource`	picklist	Channel (Web, Trade Show, Partner Referral)
`Status`	picklist	Funnel stage (use pre-cutoff value only)
`Country`/`CountryCode`, `State`/`StateCode`, `City`, `PostalCode`	string/ picklist	Geography/territory (Code variants exist when State & Country Picklists are enabled)
`Email`, `Phone`	email/ phone	Derive domain type and presence flags (not raw PII)
`LastActivityDate`, `CreatedDate`	date/ datetime	Recency/tenure features

From **Account** (for contact-based scoring): `Industry`, `NumberOfEmployees`, `AnnualRevenue`, `Type`, `Sic`, `SicDesc`, `BillingCountry`/`BillingCountryCode`, `BillingState`, `Website`, `Ownership`.

From **Contact:** `Title`, `Department`, `LeadSource`, `MailingCountry`, plus association to Account firmographics.

Do NOT use as features (post-outcome / target-defining): `IsConverted`, `ConvertedDate`, `ConvertedAccountId`, `ConvertedContactId`, `ConvertedOpportunityId`. `coda`

2B. Behavioral/Engagement Features

Marketo — activity types from the REST API `getLeadActivities` / `getAllActivityTypes` (verified canonical `activityTypeId` → name; standard IDs are stable across instances):

activityTypeId	Name	Feature
1	Visit Webpage	page-visit counts
2	Fill Out Form	form-fill counts (high intent)
3	Click Link (on web page)	web click counts
6	Send Email	emails sent (denominator)
7	Email Delivered	delivered (denominator)
8	Email Bounced	negative signal
9	Unsubscribe Email	strong negative signal
10	Open Email	open counts
11	Click Email (click link in email)	email click counts (higher intent than opens)
12	New Lead	creation event
13	Change Data Value	field-change events
46	Interesting Moment	sales-relevant signal
104	Change Status in Progression	webinar/event attendance

> Note the common misconceptions: **Email Bounced = 8** (not 10), **Unsubscribe Email = 9** (not 12), **Open Email = 10**, **New Lead = 12**. There is no standard "Attend Webinar" type — webinar/event attendance is tracked via **Change Status in Progression (104)**.

Also usable Marketo scoring fields (populated by Marketo): Lead Score (`mkto71_Lead_Score` when synced to Salesforce), plus custom Behavior Score / Demographic Score fields.

HubSpot — default contact properties (verified internal names):

Property	Feature
`hs_analytics_num_page_views`	total page views
`hs_analytics_num_visits`	sessions
`hs_analytics_num_event_completions`	events/conversions
`hs_analytics_average_page_views`	avg pages/session
`hs_email_open`	marketing emails opened
`hs_email_click`	marketing emails clicked
`hs_email_delivered`, `hs_email_bounce`	denominators / negative
`hs_analytics_source`	first-touch source
`hs_analytics_last_referrer`	latest referrer
`hs_analytics_first_timestamp`, `hs_analytics_last_timestamp`, `hs_analytics_last_visit_timestamp`	recency
`num_conversion_events`, `num_unique_conversion_events`	form conversions
`hs_sales_email_last_clicked`, `hs_sales_email_last_opened`	1:1 sales engagement
`notes_last_contacted`, `num_contacted_notes`	sales touches
`lifecyclestage`, `hs_lead_status`	funnel (pre-cutoff only)
`recent_conversion_date`, `first_conversion_date`, `first_conversion_event_name`	conversion recency/ content

HubSpot **company** properties: `num_associated_contacts`, `num_associated_deals`, `hs_num_child_companies`, `numberofemployees`, `annualrevenue`, `industry`, `hs_num_open_deals`, `num_conversion_events`.

The native predictive fields (`hs_predictivecontactscore_v2`, `hs_predictivescoringtier`, HubSpot `hubspotscore`, Marketo scores) may be used as **benchmarks/challengers**, but be cautious about using them as features (circularity/leakage from a black-box model trained on the same target).

2C. Derived / Engineered Features

- **Recency:** `days_since_last_activity` (DATEDIFF from last Marketo activity / `hs_analytics_last_timestamp` to cutoff).
- **Frequency:** counts of each activity type in trailing 7/30/90-day windows.
- **Monetary-style engagement (intensity):** weighted engagement score (form fill ×3, email click ×2, page view ×1), summed over the window.
- **Velocity:** change in activity count last 7 days vs prior 7 days (acceleration).
- **Email ratios:** open rate = opens/delivered; click-to-open = clicks/opens; bounce flag.
- **Web patterns:** distinct pages, high-intent page hits (pricing/product/"contact sales"), sessions per week.
- **Seniority band** from `Title` (C-Level/VP/Director/Manager/IC) via CASE mapping.
- **Missingness flags:** `is_revenue_missing`, `is_title_missing` — absence is itself predictive.
- **Corporate vs freemail domain** from `Email`.
- **Cyclical time** features (sin/cos of hour/day of first touch).

2D. Third-Party Enrichment Fields

- **Clearbit / Breeze Intelligence** — HubSpot documents **100+ business data attributes** drawn from 250+ data sources: company `employees`, revenue range, `category.industry`/`sector`, `tech` tags (technographics), `foundedYear`, funding (`metrics.raised`); person `role`, `seniority`, `title`. Technographic tech tags are especially useful for hardware (detecting an existing UC/video stack). Skrapp
- **ZoomInfo** — employee count, revenue range, `sicCodes`/`naicsCodes`, industry, parent/subsidiary hierarchy, plus technographics covering **more than 30,000 technologies across 200+ categories**, and intent topics (Bombora-derived). ZoomInfo
- **Dun & Bradstreet** — D-U-N-S number, employees, revenue, SIC/NAICS, corporate linkage/family tree.

3. Target Variable Definition

Choose the outcome to match the decision. Three candidate targets, in Salesforce terms:

1. **MQL → SQL:** a Lead `Status` transition into an "Accepted/Working" value.
2. **Lead → Opportunity (recommended primary):** `IsConverted = TRUE` AND `ConvertedOpportunityId` is not null within the outcome window.
3. **Opportunity → Closed Won:** on the Opportunity object, `IsWon = TRUE` (equivalently `StageName` in a Closed Won stage and `IsClosed = TRUE`).

Define an **outcome window** (e.g., 90 days, ~90th percentile of historical conversion time) and label:

```sql
-- Snowflake: label = did lead convert to opportunity within 90 days of creation
SELECT
  l.id AS lead_id,
  CASE WHEN l.isconverted = TRUE
        AND l.convertedopportunityid IS NOT NULL
        AND l.converteddate <= DATEADD(day, 90, l.createddate)
       THEN 1 ELSE 0 END AS target_converted
FROM salesforce.lead l;
```

For a Closed Won target, join `Opportunity` and use `IsWon`. Keep negatives = leads that had the opportunity to convert but did not within the window (avoid survivorship bias by not silently dropping unconverted/old leads).

4. Feature Engineering (Point-in-Time Correct)

Cardinal rule: pick a scoring cutoff timestamp per training example and compute every feature using only rows with `activity_date <= cutoff`. For historical training, the cutoff is typically lead creation (or MQL) time.

Snowflake SQL — behavioral aggregation with temporal cutoff

sql

```sql
WITH base AS (
  SELECT id AS lead_id, createddate AS cutoff_ts
  FROM salesforce.lead
),
acts AS (
  SELECT b.lead_id, a.activity_type_id, a.activity_date
  FROM base b
  JOIN marketo.activities a
    ON a.lead_id = b.lead_id
   AND a.activity_date <= b.cutoff_ts                        -- POINT-IN-TIME CUTOFF
   AND a.activity_date >  DATEADD(day, -90, b.cutoff_ts)     -- 90-day window
)
SELECT
  b.lead_id,
  COUNT_IF(a.activity_type_id = 1)  AS page_visits_90d,
  COUNT_IF(a.activity_type_id = 2)  AS form_fills_90d,
  COUNT_IF(a.activity_type_id = 10) AS email_opens_90d,
  COUNT_IF(a.activity_type_id = 11) AS email_clicks_90d,
  COUNT_IF(a.activity_type_id = 9)  AS unsubscribes_90d,
  DATEDIFF(day, MAX(a.activity_date), b.cutoff_ts) AS days_since_last_activity,
  SUM(CASE a.activity_type_id WHEN 2 THEN 3 WHEN 11 THEN 2 WHEN 1 THEN 1 ELSE 0 END)
FROM base b
LEFT JOIN acts a ON a.lead_id = b.lead_id
GROUP BY b.lead_id, b.cutoff_ts;
```

Velocity feature (7-day vs prior 7-day)

sql

```sql
SELECT
  b.lead_id,
  COUNT_IF(a.activity_date >  DATEADD(day,-7,  b.cutoff_ts)) AS acts_last_7d,
  COUNT_IF(a.activity_date <= DATEADD(day,-7,  b.cutoff_ts)
           AND a.activity_date > DATEADD(day,-14, b.cutoff_ts)) AS acts_prior_7d,
  (COUNT_IF(a.activity_date > DATEADD(day,-7, b.cutoff_ts))
   - COUNT_IF(a.activity_date <= DATEADD(day,-7, b.cutoff_ts)
              AND a.activity_date > DATEADD(day,-14, b.cutoff_ts))) AS activity_veloc
FROM base b
LEFT JOIN marketo.activities a
  ON a.lead_id = b.lead_id AND a.activity_date <= b.cutoff_ts
GROUP BY b.lead_id, b.cutoff_ts;
```

Seniority band from Title (Snowflake)

```sql
SELECT id AS lead_id,
  CASE
    WHEN title ILIKE ANY ('%chief%','%ceo%','%cfo%','%cto%','%cio%','% vp%','%vice pr
    WHEN title ILIKE ANY ('%director%','%head of%') THEN 'director'
    WHEN title ILIKE '%manager%' THEN 'manager'
    WHEN title IS NULL THEN 'unknown'
    ELSE 'individual_contributor'
  END AS seniority_band
FROM salesforce.lead;
```

Categorical encoding (Python)

```python
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
from category_encoders import TargetEncoder  # high-cardinality

# Low-cardinality (Industry, LeadSource, seniority_band, CountryCode): one-hot
low_card = ['Industry','LeadSource','seniority_band','CountryCode']
ohe = OneHotEncoder(handle_unknown='ignore', min_frequency=20, sparse_output=False)

# High-cardinality (State, sic_code): target encoding WITHIN CV folds to avoid leakag
high_card = ['State','sic_code']
te = TargetEncoder(cols=high_card, smoothing=10)   # fit on training folds only
```

Rules: fit encoders on the training split only; `handle_unknown='ignore'`; group rare levels (`min_frequency`); target-encode inside CV folds so a row's own target never informs its encoding.

Missing values

```python
# Tree models (XGBoost/LightGBM) handle NaN natively — keep NaN + add missingness fla
for col in ['AnnualRevenue','NumberOfEmployees','Title']:
    df[f'{col}_is_missing'] = df[col].isna().astype(int)
# For the logistic-regression baseline: median-impute numerics, 'missing' category fo
from sklearn.impute import SimpleImputer
num_imp = SimpleImputer(strategy='median')
```

Leakage checklist

- Exclude `IsConverted`, `Converted*`, `IsWon`, `IsClosed`, `StageName`, `closedate`, native `hs_predictive*` (if trained on the same label), and any `Status` value set only post-conversion.
- Enforce `activity_date <= cutoff` everywhere.
- Use a **temporal train/test split** (e.g., train months 1–12, test months 13–15), never a random split.
- Any feature with **Information Value > 0.5 → investigate for leakage**.

5. Feature Selection

Filter methods

python

```python
import numpy as np, pandas as pd
from sklearn.feature_selection import mutual_info_classif, chi2, VarianceThreshold

# 1) Variance threshold — drop near-constant features
vt = VarianceThreshold(threshold=0.0)

# 2) Mutual information (captures non-linear relationships)
mi = mutual_info_classif(X, y, discrete_features='auto', random_state=42)
mi_s = pd.Series(mi, index=X.columns).sort_values(ascending=False)

# 3) Chi-square for non-negative categorical/count features
chi_stat, p = chi2(X_nonneg, y)

# 4) Information Value (WOE) — classic in scoring
def iv_woe(df, feature, target, bins=10):
    d = df[[feature, target]].copy()
    d['bin'] = pd.qcut(d[feature].rank(method='first'), bins, duplicates='drop')
    g = d.groupby('bin')[target].agg(['count','sum'])
    g['non_event'] = g['count'] - g['sum']
    g['pct_event'] = g['sum']/g['sum'].sum()
    g['pct_non']   = g['non_event']/g['non_event'].sum()
    g['woe'] = np.log((g['pct_event']+1e-6)/(g['pct_non']+1e-6))
    g['iv']  = (g['pct_event']-g['pct_non'])*g['woe']
    return g['iv'].sum()
# IV guide: <0.02 useless | 0.02-0.1 weak | 0.1-0.3 medium | 0.3-0.5 strong | >0.5 su
```

Wrapper — Recursive Feature Elimination

```python
from sklearn.feature_selection import RFECV
from sklearn.linear_model import LogisticRegression
rfe = RFECV(LogisticRegression(penalty='l2', max_iter=1000, class_weight='balanced'),
            step=1, cv=5, scoring='roc_auc', min_features_to_select=8)
rfe.fit(X_train, y_train)
selected = X_train.columns[rfe.support_]
```

Embedded — L1/Lasso and tree importance

```python
from sklearn.linear_model import LogisticRegression
l1 = LogisticRegression(penalty='l1', solver='liblinear', C=0.1, class_weight='balanc
l1.fit(X_train, y_train)
lasso_selected = X_train.columns[(l1.coef_ != 0).ravel()]

import lightgbm as lgb
gbm = lgb.LGBMClassifier(n_estimators=400).fit(X_train, y_train)
imp = pd.Series(gbm.feature_importances_, index=X_train.columns).sort_values(ascendin

# permutation importance is more reliable than Gini for correlated/high-cardinality f
from sklearn.inspection import permutation_importance
perm = permutation_importance(gbm, X_val, y_val, scoring='roc_auc', n_repeats=10, ran
```

Redundancy: drop |corr| > 0.9; check VIF > 10.

SHAP (explainability + selection)

```python
import shap
explainer = shap.TreeExplainer(gbm)
shap_values = explainer.shap_values(X_val)
shap.summary_plot(shap_values, X_val)                    # global beeswarm
shap.summary_plot(shap_values, X_val, plot_type="bar") # mean |SHAP| importance
# per-lead explanation for reps ("top reasons"), mirroring Einstein top positives/neg
shap.force_plot(explainer.expected_value, shap_values[i], X_val.iloc[i])
```

Feature-count rule of thumb: ~10–20 positive outcomes per feature. For mid-market volumes, 5–8 up to ~30 strong features is typical; fewer, cleaner features often beat many noisy ones.

6. Modeling

Baseline — regularized logistic regression

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
logit = Pipeline([('sc', StandardScaler(with_mean=False)),
                  ('clf', LogisticRegression(class_weight='balanced', max_iter=1000,
```

Primary — LightGBM / XGBoost

```python
import lightgbm as lgb
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import roc_auc_score

neg, pos = (y_train==0).sum(), (y_train==1).sum()
spw = neg / pos    # class-imbalance weight ~ n_negative / n_positive

params = dict(
    objective='binary', n_estimators=2000, learning_rate=0.03,
    num_leaves=31, max_depth=-1, min_child_samples=50,
    subsample=0.8, colsample_bytree=0.8,
    reg_alpha=1.0, reg_lambda=1.0, scale_pos_weight=spw, random_state=42)
model = lgb.LGBMClassifier(**params)
model.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], eval_metric='auc',
          callbacks=[lgb.early_stopping(100), lgb.log_evaluation(0)])
```

The XGBoost equivalent uses `scale_pos_weight = neg/pos`, `eval_metric='aucpr'`, and `early_stopping_rounds`.

Class imbalance

Prefer `scale_pos_weight` / `class_weight='balanced'` plus threshold tuning. **SMOTE is optional and risky:** resampling distorts probability calibration; if used, apply it only to the training fold inside an `imblearn.pipeline` (never validation/test), and recalibrate afterward.

Hyperparameter tuning (time-aware CV)

```python
import optuna
def objective(trial):
    p = dict(objective='binary', n_estimators=3000,
             learning_rate=trial.suggest_float('lr',0.01,0.1,log=True),
             num_leaves=trial.suggest_int('leaves',15,127),
             min_child_samples=trial.suggest_int('mcs',20,200),
             subsample=trial.suggest_float('ss',0.6,1.0),
             colsample_bytree=trial.suggest_float('cs',0.6,1.0),
             reg_lambda=trial.suggest_float('l2',1e-3,10,log=True),
             scale_pos_weight=spw)
    aucs=[]
    for tr_idx, va_idx in TimeSeriesSplit(5).split(X):
        m = lgb.LGBMClassifier(**p).fit(
             X.iloc[tr_idx], y.iloc[tr_idx],
             eval_set=[(X.iloc[va_idx], y.iloc[va_idx])], eval_metric='auc',
             callbacks=[lgb.early_stopping(80)])
        aucs.append(roc_auc_score(y.iloc[va_idx], m.predict_proba(X.iloc[va_idx])[:,1
    return np.mean(aucs)
study = optuna.create_study(direction='maximize'); study.optimize(objective, n_trials
```

Calibration (lead scores MUST be calibrated probabilities)

```python
from sklearn.calibration import CalibratedClassifierCV
# fit the calibrator on a held-out calibration set the base model did NOT train on
cal = CalibratedClassifierCV(model, method='isotonic', cv='prefit')
cal.fit(X_calib, y_calib)   # 'isotonic' (flexible, needs data) or 'sigmoid' (Platt,
proba = cal.predict_proba(X_test)[:,1]
```

Isotonic regression typically gives the strongest calibration improvement when data is sufficient; Platt (sigmoid) is safer for small samples. Always verify with a reliability curve and Brier score, and never calibrate on the model's own training data.

7. Evaluation Metrics

```python
from sklearn.metrics import roc_auc_score, average_precision_score, brier_score_loss
auc    = roc_auc_score(y_test, proba)           # rank quality; aim >=0.75, strong >
pr_auc = average_precision_score(y_test, proba) # better under heavy imbalance
brier  = brier_score_loss(y_test, proba)        # calibration/accuracy
```

- **AUC-ROC** — overall discrimination.
- **PR-AUC / average precision** — prioritize under class imbalance.
- **Lift & decile/gains analysis** — business-facing: sort by score, bucket into deciles, measure conversion capture. A good marketing model shows **top-decile lift around 3× or higher**. K2 Analytics
- **Precision@k** — of the top-k leads reps can realistically work, what fraction convert.
- **Calibration curve + Brier score** — predicted 30% should convert ~30%.

```python
# decile lift table
df_eval = pd.DataFrame({'y': y_test.values, 'p': proba})
df_eval['decile'] = pd.qcut(df_eval['p'].rank(method='first'), 10, labels=range(10,0,
lift = (df_eval.groupby('decile')['y'].agg(['count','sum'])
        .assign(rate=lambda d: d['sum']/d['count']))
lift['lift'] = lift['rate'] / df_eval['y'].mean()
```

8. Deployment in Salesforce + Marketo/HubSpot + Snowflake

- **Feature pipeline:** schedule the Snowflake SQL (Snowflake Tasks or dbt) to materialize a point-in-time feature table; score in Python (Snowpark or an external job).
- **Write-back to Salesforce:** create custom fields on Lead/Contact — e.g., `Predictive_Lead_Score__c` (Number, 0–100 or probability), `Score_Tier__c` (picklist), and `Score_Top_Reasons__c` (text, from SHAP). Update via Salesforce Bulk API 2.0 / `simple_salesforce`.

```python
from simple_salesforce import Salesforce
sf = Salesforce(username=..., password=..., security_token=...)
records = [{'Id': r.lead_id, 'Predictive_Lead_Score__c': float(r.score)} for r in sco
sf.bulk.Lead.update(records)   # batch write-back
```

- **Scoring cadence:** batch daily or every 6 hours for the active funnel (matches Einstein's "at least every six hours" refresh); near-real-time on high-intent events (e.g., a demo request) via a trigger.
- **Monitoring & retraining:** track score distribution weekly; MQL→SQL acceptance and top-decile conversion monthly; recalibrate quarterly. Monitor input drift with **PSI — a value > 0.2 triggers retraining**; also retrain after product launches, pricing changes, or ICP shifts. B2B models can degrade within 90–180 days. Run **shadow scoring** (score in parallel without routing) before cutover.

9. Common Pitfalls

- **Leakage from post-conversion fields:** never feed `IsConverted`, `ConvertedOpportunityId`, `StageName`, `IsWon`, `closedate`, or Status values set only after conversion.
- **Temporal leakage:** using activities after the cutoff; always enforce point-in-time features and temporal splits.
- **Survivorship bias:** training only on leads still in the system; include leads that went cold.
- **Sales feedback loops:** if reps only work high-scored leads, only those get a chance to convert, biasing the next model — mitigate with a holdout/exploration group and by modeling the whole funnel.
- **Class-imbalance mishandling:** optimizing accuracy (meaningless at a 2% base rate); use PR-AUC/lift and calibration; avoid resampling the test set.
- **Data quality:** practitioners recommend fields be ≥70% complete; enrich or drop otherwise. Standardize categorical values (e.g., "Financial Services" vs "Fin Svcs"; State abbreviations).

Recommendations (staged)

1. **Weeks 1–3 — Foundation:** land Salesforce, Marketo/HubSpot, and enrichment into Snowflake; define the target (Lead→Opportunity, 90-day window); build the point-in-time feature table; audit completeness (drop <70% fields or enrich).
2. **Weeks 4–6 — Baseline:** logistic regression with filter + L1 selection; temporal split; establish AUC/PR-AUC/lift baselines. **Proceed only if AUC ≥ 0.75.**
3. **Weeks 7–9 — Primary model:** LightGBM/XGBoost with `scale_pos_weight`, Optuna tuning, isotonic calibration, and SHAP reasons. Target **AUC ≥ 0.80 and top-decile lift ≥ 3×**.
4. **Weeks 10–12 — Deploy:** write `Predictive_Lead_Score__c` + tier + reasons back to Salesforce; shadow-score for 4–6 weeks against the current process before routing on it.
5. **Ongoing:** weekly distribution checks, monthly conversion review, quarterly recalibration; retrain when PSI > 0.2 or after major GTM changes.

Thresholds that change the plan: AUC < 0.70 after feature work → revisit the target/data quality; do not deploy. More than 20% of Closed Won deals falling below the MQL threshold → the model is missing signals; add features/retrain. Worsening Brier calibration or PSI > 0.2 → retrain.

Caveats

- Native predictive scores (`hs_predictivecontactscore_v2`, Einstein Lead Score, Marketo scores) are black-box; use them as challenger benchmarks, not ground truth, and avoid them as features when trained on the same label.
- Marketo **custom** activity type IDs (100000+) are instance-specific; confirm your instance's IDs via `getAllActivityTypes`. Standard IDs (1, 2, 10, 11, etc.) are stable. Informatica
- The standalone Clearbit enrichment offering has been folded into HubSpot **Breeze Intelligence** (HubSpot acquired Clearbit in December 2023; free standalone Clearbit tools sunset April 30, 2025, and the Logo API ended December 1, 2025). Confirm your current access path before building on it. Pixelodigital GlobalDatabase
- Enrichment coverage thins outside North America and for SMB; technographic signals reflect **detection**, not confirmed usage.
- Some vendor performance claims (e.g., pipeline-lift case studies) are vendor-reported and not independently verified.